
中山大学计算机院本科生实验报告

(2023 学年春季学期)

课程名称：并行程序设计

批改人：

实验	9-CUDA 并行矩阵乘法	专业（方向）	计算机科学与技术计科一班
学号	21307099	姓名	李英骏
Email	liyj323@mail2.sysu.edu.cn	完成日期	2024 年 6 月 2 日

目录

1	实验目的	2
2	实验过程和核心代码	2
2.1	文件结构	2
2.1.1	Naive	3
2.1.2	分块乘法	4
2.1.3	向量化访存	10
2.1.4	Bank Conflict	10
2.1.5	Double Buffer	12
3	实验结果	13
4	实验感想	19

1 实验目的

1. CUDA 实现并行通用矩阵乘法
2. 并通过实验分析不同线程块大小, 访存方式、数据/任务划分方式对并行性能的影响

2 实验过程 and 核心代码

2.1 文件结构

```
> tree ./
./
├── build_run.sh
├── include
│   ├── kernels.cuh
│   └── utils.hpp
├── kernels
│   ├── blockGEMM.cu
│   ├── conflictFreeGEMM.cu
│   ├── doubleBufferGEMM.cu
│   ├── naiveGEMM.cu
│   └── vectorizeGEMM.cu
├── main.cu
├── test.cu
├── test.sh
├── test_block_16.txt
├── test_block_8.txt
├── utils
│   ├── mat_utils.cu
│   └── test_utils.cu
└── 3 directories, 15 files
~/parallel_programming/LAB10
```

图 1: tree

运行 build_run.sh 即可编译和运行代码, 将 main.cu 参与编译, 生成符合要求输入输出的 main 可执行文件

运行 test.sh 将 test.cu 参与编译, 生成 test_block_8.txt 和 test_block_16.txt, 分别为 BLOCKSIZE=8 和 16 的计时和测试结果.

kernels 文件夹是实现的核函数, utils 文件夹是一些工具函数.

2.1.1 Naive

$$A_{M \times N} B_{N \times K} = C_{M \times K}$$

在每个线程中计算结果矩阵 C 的一个元素 $C_{m,k} = \sum_{n=0}^{N-1} A_{m,n} B_{n,k}$

```
kernels > g naiveGEMM.cu > naiveGEMM(FLOAT_TYPE __restrict__ FLOAT_TYPE __restrict__ FLOAT_TYPE __restrict__ const int, const int, const int)
1  #include "../include/kernels.cuh"
2
3  __global__ void naiveGEMM(FLOAT_TYPE __restrict__ a, FLOAT_TYPE __restrict__ b, FLOAT_TYPE __restrict__ c, const int M,
4  const int N, const int K)
5  {
6      int m = blockIdx.y * blockDim.y + threadIdx.y;
7      int k = blockIdx.x * blockDim.x + threadIdx.x;
8      if (m < M && k < K)
9      {
10         float psum = 0.0f;
11         #pragma unroll
12         for (int n = 0; n < N; n++)
13         {
14             psum += a[OFFSET(m,n,N)] * b[OFFSET(n,k,K)];
15         }
16         c[OFFSET(m,k,K)] = psum;
17     }
18 }
```

图 2: Naive

若每个 BLOCK SIZE 为 $B_X \times B_Y$, 则 GRID SIZE 为 $\lceil \frac{M}{B_X} \rceil \times \lceil \frac{K}{B_Y} \rceil$

访存分析

每个元素的计算需要读取 A 的整行和 B 的整列, 即共 $2MKN$ 次 GMEM(全局内存) 读, MK 次 GMEM 写.

2.1.2 分块乘法

1. 先考虑 GMEM 到 SMEM(共享内存) 的分块:

将小矩阵块先存储到 SMEM 中, 而后计算时可以直接从 SMEM 中取数, 从而减少访存时延. 同时, 该算法可以显著地减少对 GMEM 的访存次数.

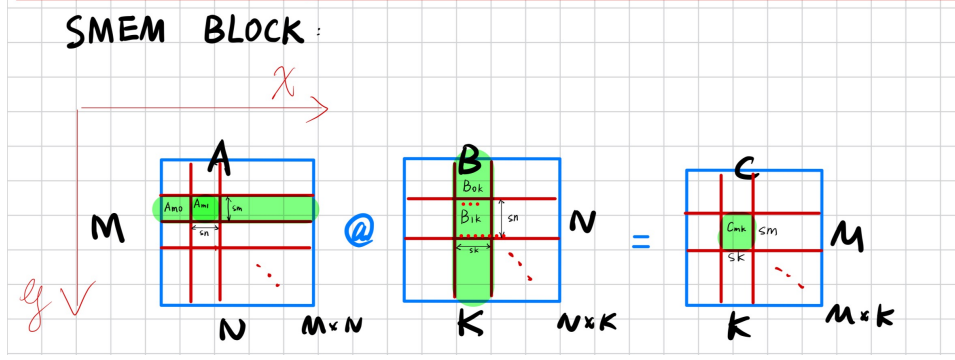


图 3: MEM BLOCK

如上图, 我们将 A 分成 $sm \times sn$ 的 tiles, B 分成 $sn \times sk$ 的 tiles, 则最后的 C 矩阵分成 $sm \times sk$ 的 tiles. 即

$$A: M \times N = (M_ \cdot sm) \times (N_ \cdot sn) \quad (1)$$

$$B: N \times K = (N_ \cdot sn) \times (K_ \cdot sk) \quad (2)$$

$$C: M \times K = (M_ \cdot sm) \times (K_ \cdot sk) \quad (3)$$

每个 BLOCK 计算 C 的一个子矩阵 $C_{m,k}$, 则共 $M_ \cdot K_$ 个 BLOCK. 每个 BLOCK 执行以下计算:

$$C_{m,k} = \sum_{n=0}^{N_ - 1} A_{mn} B_{nk} \quad (4)$$

其中 A_{mn} 和 B_{nk} 都是子矩阵. 显然, GRID SIZE 为 $M_ \times K_ = \frac{M}{sm} \times \frac{K}{sk}$

访存分析

每个 BLOCK 需要读取 $N_ \cdot (sm \cdot sn + sn \cdot sk)$ 次 GMEM, 并写入 GMEM. 即读取 GMEM 的次数为:

$$M_ \cdot K_ \cdot N_ \cdot (sm \cdot sn + sn \cdot sk) \quad (5)$$

$$= MNK \left(\frac{1}{sm} + \frac{1}{sk} \right) \quad (6)$$

即 sm 和 sk 越大, 对 GMEM 的访存次数越少.

2. 然后考虑 SMEM 到 Register 的分块:

同样地, 由于对 Register 的访问速度远快于 SMEM, 我们可以将 SMEM 中的数据分块到 Register 中, 从而减少访存次数和访存时延.

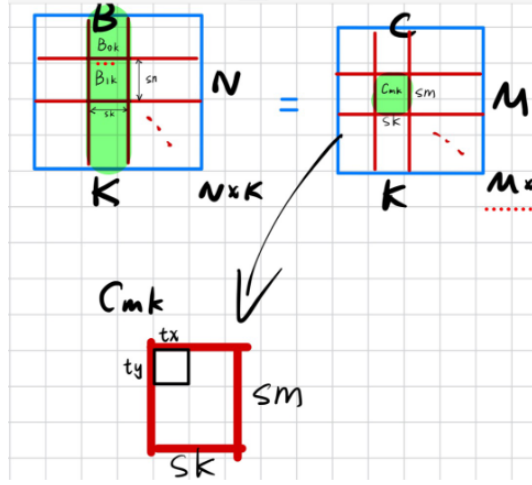


图 4: SEMM2REG

用以下算法每个线程中计算一个 tile 中的 $t_x \times t_y$ 个元素:

$$C = \sum_{i=0}^{sn-1} C_i = \sum_{i=0}^{sn-1} A_i (B_i^T)^T = \sum_{i=0}^{sn-1} \begin{bmatrix} a_{1i}b_{i1} & a_{1i}b_{i2} & \cdots & a_{1i}b_{in} \\ a_{2i}b_{i1} & a_{2i}b_{i2} & \cdots & a_{2i}b_{in} \\ \vdots & \vdots & \ddots & \vdots \\ a_{mi}b_{i1} & a_{mi}b_{i2} & \cdots & a_{mi}b_{in} \end{bmatrix} \quad (7)$$

其中 C 由 sn 个 C_i 的和组成, A_i 是 A 的第 i 列, $(B_i^T)^T$ 是 B 的第 i 行, 它们做外积.

访存分析

若不对 SMEM 分块, 则每个 BLOCK 需要负责计算一个 tile, BLOCK 中的每个线程负责计算 tile 中的单个元素. 需要读取 $2sm \cdot sk \cdot sn$ 次 SMEM.

我们对 SMEM 分块后, 每一块大小为 $t_x \times t_y$, 则 BLOCK 中的每个 thread 负责计算一个 tile 中的 $t_x \times t_y$ 个元素.

此时 BLOCK 中的线程数量 (BLOCK SIZE) 为 $\frac{sm}{t_y} \times \frac{sk}{t_x}$, 每个 BLOCK 需要读取

$$\frac{sm}{t_y} \times \frac{sk}{t_x} \cdot (t_x + t_y)sn \quad (8)$$

$$= sm \cdot sk \cdot sn \left(\frac{1}{t_x} + \frac{1}{t_y} \right) \quad (9)$$

次 SMEM.

3. 考虑上述操作中的约束条件:

我们安排每个 thread 计算一个 tile 中 $tx \times ty$ 个元素. 由于每个线程 BLOCK 计算 C 的一个大小为 $sm \times sk$ 的子矩阵 $C_{m,k}$, 一个 BLOCK 中的线程数量 (BLOCK SIZE) 为 $\frac{sm}{ty} \times \frac{sk}{tx}$:

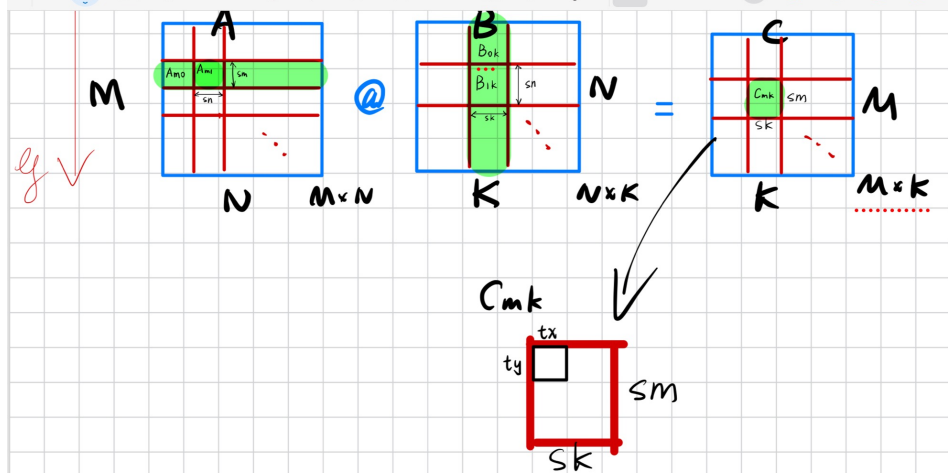


图 5: THREAD

然后需要考虑到几个限制 (如下图):

1. 每个 BLOCK 的线程数量有限:

$$\frac{sm}{ty} \times \frac{sk}{tx} \leq 1024 \quad (10)$$

2. 每个 BLOCK 的 SMEM 容量有限.

考虑到后面的 double buffer:

$$4 \text{ Bytes} \times (sm \cdot sn + sn \cdot sk) \leq 49152/2 \text{ Bytes} = 24576 \text{ Bytes} \quad (11)$$

3. 每个 BLOCK 的寄存器数量有限.

考虑到 Gmem to Smem 和 Smem to Reg 需要一些寄存器, 留一半的寄存器给其他用途, 则

$$sm \times sk \leq 65536/2 = 128 \times 256 \quad (12)$$

```
> ./test
Device 0: NVIDIA GeForce RTX 4070
Total number of SMs: 46
Maximum number of threads per SM: 1536
Maximum number of threads per block: 1024
Maximum size of each dimension of a block: 1024 x 1024 x 64
Maximum size of each dimension of a grid: 2147483647 x 65535 x 65535
Shared memory per block: 49152 bytes
Total global memory: 11.9937 GB
Number of registers per SM: 65536
Number of registers per block: 65536
Maximum registers per thread: 64
```

图 6: GPU: RTX4070

LDS.128 指令每次读 4 个 float32, 并且为了尽可能减少迭代次数 sn 不应太小, 故 sn 可以取 8 或 16. 再考虑到上面 sm 和 sk 越大, 对 GMEM 的访存次数越少. 并且测试矩阵中有 128×128 , 即 sm 和 sk 不能大于 128.

我们使用 Cuda_Occupancy_Calculator 计算 Occupancy 最佳的 BLOCK SIZE 和 GRID SIZE:

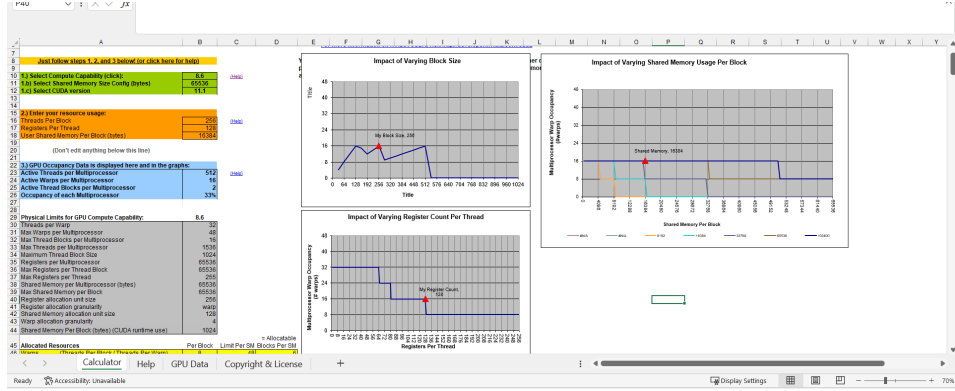


图 7: CUDA Occupancy Calculator

可以看出两组理论上可能最佳的参数 (在计算能力 8.6 上 Occupancy 为 33%, 7.5 上为 50%):

1. $sm = 128, sn = 16, sk = 128, tx = 8, ty = 8$

此时 BLOCK SIZE 为 $\frac{sm}{ty} \times \frac{sk}{tx} = 16 \times 16$, GRID SIZE 为 $\frac{M}{128} \times \frac{K}{128}$, 每个线程存计算结果占用的寄存器数量为 $8 \times 8 = 64$, 每个 BLOCK 占用 16284 Bytes.

2. $sm = 128, sn = 8, sk = 128, tx = 8, ty = 8$

此时 BLOCK SIZE 为 $\frac{sm}{ty} \times \frac{sk}{tx} = 16 \times 16$, GRID SIZE 为 $\frac{M}{128} \times \frac{K}{128}$, 每个线程存计算结果占用的寄存器数量为 $8 \times 8 = 64$, 每个 BLOCK 占用 8192 Bytes.

其他可取的 BLOCK SIZE(对应不同的任务划分) 在上述不等式10,11,12的限制下. 为方便我们选取正方形的 BLOCK 和 THREAD, 则其他可以取的参数为:

1. $sm = 64, sn = 8, sk = 64, tx = 8, ty = 8$

此时 BLOCK SIZE 为 $\frac{sm}{ty} \times \frac{sk}{tx} = 8 \times 8$, GRID SIZE 为 $\frac{M}{64} \times \frac{K}{64}$, 每个线程存计算结果占用的寄存器数量为 $8 \times 8 = 64$, 每个 BLOCK 占用 4096 Bytes.

2. $sm = 64, sn = 16, sk = 64, tx = 8, ty = 8$

此时 BLOCK SIZE 为 $\frac{sm}{ty} \times \frac{sk}{tx} = 8 \times 8$, GRID SIZE 为 $\frac{M}{64} \times \frac{K}{64}$, 每个线程存计算结果占用的寄存器数量为 $8 \times 8 = 64$, 每个 BLOCK 占用 8192 Bytes.

分块乘法的代码如下 (仅展示 sn=8 的 ./kernels/blockGEMM.cu):

```

1 // 分块乘法
2
3 __global__ void blockGEMM_sn8(
4     float * __restrict__ a, float * __restrict__ b, float * __restrict__ c,
5     const int M, const int N, const int K)
6 {
7     // BLOCK = sm/ty * sk/tx
8     constexpr int sm_ty = BLOCK_SIZE;
9     constexpr int sk_tx = BLOCK_SIZE;
10    // 可变的sn
11    constexpr int sn = 8;
12    // 每个线程负责计算的子矩阵大小tx*ty
13    constexpr int tx = 8;
14    constexpr int ty = 8;
15    // sm = BLOCK_SIZE * ty, sk = BLOCK_SIZE * tx
16    constexpr int sm = sm_ty * ty;
17    constexpr int sk = sk_tx * tx;
18    // 计算BLOCK的索引
19    // const int M_ = M / sm;
20    const int N_ = N / sn;
21    // const int K_ = K / sk;
22    const int B_x = blockIdx.x;
23    const int B_y = blockIdx.y;
24    const int T_x = threadIdx.x;
25    const int T_y = threadIdx.y;
26    // 计算线程的索引
27    const int thread_index = T_y * blockDim.x + T_x;
28    // SMEM
29    __shared__ float A_smem[sm][sn];
30    __shared__ float B_smem[sn][sk];
31    // Register per thread
32    float C_reg[ty][tx] = {0.0f};
33
34    // 计算从GMEM中加载到SMEM的索引
35    // 一个BLOCK加载A的sm*sn, 共BLOCK_SIZE*BLOCK_SIZE个线程
36    // 每个线程负责加载A的sm*sn/BLOCK_SIZE^2 = sn*ty/BLOCK_SIZE个元素
37    constexpr int read_per_thread = sn * ty / BLOCK_SIZE;
38    const int a_smem_y = thread_index * ty / BLOCK_SIZE;
39    const int a_smem_x = (thread_index * read_per_thread) % sn;
40    const int b_smem_y = thread_index * read_per_thread / sk;
41    const int b_smem_x = (thread_index * read_per_thread) % sk;
42    // 计算从GMEM加载的固定index
43    const int a_gmem_y = B_y * sm + a_smem_y;
44    const int b_gmem_x = B_x * sk + b_smem_x;
45
46    for (int n = 0; n < N_; ++n)
47    {
48        // 把数据从GMEM加载到SMEM
49        // 计算与n相关的index
50        int a_gmem_x = n * sn + a_smem_x;
51        int b_gmem_y = n * sn + b_smem_y;
52        int a_gmem_addr = OFFSET(a_gmem_y, a_gmem_x, N);
53        int b_gmem_addr = OFFSET(b_gmem_y, b_gmem_x, K);
54
55        #pragma unroll
56        for (int i = 0; i < read_per_thread; ++i)
57        {
58            A_smem[a_smem_y][a_smem_x + i] = a[a_gmem_addr + i];
59            B_smem[b_smem_y][b_smem_x + i] = b[b_gmem_addr + i];
60        }
61        __syncthreads();
62
63        // 计算C_reg
64        #pragma unroll
65        for (int tn = 0; tn < sn; ++tn)
66        {
67            #pragma unroll
68            for (int tm = 0; tm < ty; ++tm)
69            {
70                #pragma unroll
71                for (int tk = 0; tk < tx; ++tk)
72                {
73                    // 每次加载A的一列和B的一行
74                    int a_smem_addr = T_y * ty + tm;
75                    int b_smem_addr = T_x * tx + tk;
76                    C_reg[tm][tk] += A_smem[a_smem_addr][tn] * B_smem[tn][b_smem_addr];
77                }
78            }
79            __syncthreads();
80        }
81
82        // 把C_reg写回GMEM
83        for (int ri = 0; ri < ty; ++ri)
84        {
85            int C_gmem_y = B_y * sm + T_y * ty + ri;
86            #pragma unroll
87            for (int rj = 0; rj < tx; rj += 4)
88            {
89                int C_gmem_x = B_x * sk + T_x * tx + rj;
90                int C_gmem_addr = OFFSET(C_gmem_y, C_gmem_x, K);
91                C[C_gmem_addr] = C_reg[ri][rj];
92                C[C_gmem_addr+1] = C_reg[ri][rj+1];
93                C[C_gmem_addr+2] = C_reg[ri][rj+2];
94                C[C_gmem_addr+3] = C_reg[ri][rj+3];
95            }
96        }
97    }
98 }

```

图 8: Block Matmul

BLOCKSIZE 只取 8 和 16 的说明

一个 BLOCK 负责加载 A 的 $sm * sn$ 部分到 A_SMEM, 一个 BLOCK 中的线程数量为 $BLOCKSIZE * BLOCKSIZE$. 则每个线程负责加载:

$$\frac{sm * sn}{BLOCKSIZE * BLOCKSIZE} \uparrow \text{float} \quad (13)$$

我们限定每个线程处理 $8*8$ 的 tile, 则:

$$sm = 8 \times BLOCKSIZE \quad (14)$$

而为了进行后面的矩阵乘法不产生过多 bank conflict(同时若增加算法从多行中取数, 会使得寄存器超过 128 个, 降低 occupancy), 每次取到 A_SMEM 的元素不能超过 A_SMEM 的一行, 即:

$$\frac{sm * sn}{BLOCKSIZE * BLOCKSIZE} = \frac{8sn}{BLOCKSIZE} \leq sn \quad (15)$$

从而:

$$BLOCKSIZE \geq 8 \quad (16)$$

另一方面, 打开 `-ptxas-options=-v`, 可以看到每个 kernel 函数使用的寄存器数量大致为 127-128:

```
ptxas info      : Function properties for '_Z14blockGEMV_sn8PfS_iii'
0 bytes stack frame, 0 bytes spill stores, 0 bytes spill loads
ptxas info      : Used 128 registers, 8192 bytes smem, 388 bytes cmem[0]
ptxas info      : Compiling entry function '_Z13blockGEMV_sn8PfS_iii' for 'sm_89'
ptxas info      : Function properties for '_Z13blockGEMV_sn8PfS_iii'
0 bytes stack frame, 0 bytes spill stores, 0 bytes spill loads
ptxas info      : Used 128 registers, 8192 bytes smem, 388 bytes cmem[0]
ptxas info      : Overriding maximum register limit 256 for '_Z16conflictFreeGEMMPfS_iii' with 200 of maxrregcount option
ptxas info      : 0 bytes gmem
ptxas info      : Compiling entry function '_Z16conflictFreeGEMMPfS_iii' for 'sm_89'
ptxas info      : Function properties for '_Z16conflictFreeGEMMPfS_iii'
0 bytes stack frame, 0 bytes spill stores, 0 bytes spill loads
ptxas info      : Used 127 registers, 16384 bytes smem, 388 bytes cmem[0]
ptxas info      : Overriding maximum register limit 256 for '_Z9naiveGEMMPfS_iii' with 200 of maxrregcount option
ptxas info      : 0 bytes gmem
ptxas info      : Compiling entry function '_Z9naiveGEMMPfS_iii' for 'sm_89'
```

图 9: reg

当 BLOCKSIZE 为 32 时, 每个 BLOCK 的寄存器数量为 $32 * 32 * 128 = 131072 > 65536$, 超过了寄存器数量的限制. 再考虑整除, BLOCKSIZE 只能取 8 或 16.

2.1.3 向量化访存

把上述的访存改用 float4, 告诉编译器用向量化的 LDG.128 和 STG.128 指令一次读 4 个 float32: 右边是普通的访存, 左边是向量化访存:

```

52 for (int index = 0; index < read_per_thread; index += 4)
53 {
54     FLOAT4(A_smem[a_smem_y][a_smem_x + index]) = FLOAT4(a
55         [a_gmem_addr + index]);
56     FLOAT4(B_smem[b_smem_y][b_smem_x + index]) = FLOAT4(b
57         [b_gmem_addr + index]);
58 }
59 __syncthreads();
60
61
62 int b_gmem_addr = OFFSET(b_gmem_y, b_gmem_x, K);
63 #pragma unroll
64 for (int i = 0; i < read_per_thread; ++i)
65 {
66     A_smem[a_smem_y][a_smem_x + i] = a[a_gmem_addr + i];
67     B_smem[b_smem_y][b_smem_x + i] = b[b_gmem_addr + i];
68 }
69 __syncthreads();
70

```

图 10: Vec Read

```

78 // 把C_reg写回GMEM
79 for (int ri = 0; ri < ty; ++ri)
80 {
81     int C_gmem_y = B.y * sm + T.y * ty + ri;
82     #pragma unroll
83     for (int rj = 0; rj < tx; rj += 4)
84     {
85         int C_gmem_x = B.x * sk + T.x * tx + rj;
86         int C_gmem_addr = OFFSET(C_gmem_y, C_gmem_x, K);
87         FLOAT4(c[C_gmem_addr]) = FLOAT4(C_reg[ri][rj]);
88     }
89 }
90
91
92 // 把C_reg写回GMEM
93 for (int ri = 0; ri < ty; ++ri)
94 {
95     int C_gmem_y = B.y * sm + T.y * ty + ri;
96     #pragma unroll
97     for (int rj = 0; rj < tx; rj += 4)
98     {
99         int C_gmem_x = B.x * sk + T.x * tx + rj;
100         int C_gmem_addr = OFFSET(C_gmem_y, C_gmem_x, K);
101         c[C_gmem_addr+1] = C_reg[ri][rj+1];
102         c[C_gmem_addr+2] = C_reg[ri][rj+2];
103         c[C_gmem_addr+3] = C_reg[ri][rj+3];
104     }
105 }

```

图 11: Vec Write

2.1.4 Bank Conflict

我们在取 A 来计算时取的是它的一列, 这样显然会有 bank conflict, A 的列数是 128 的倍数. 因此我们可以把 A 转置地存入 SMEM, 并且切块处理 BLOCK 负责的矩阵 (如下图), 这样可以避免 SMEM 的 bank conflict.

register 的 bank conflict 似乎要手动调寄存器映射, 不做处理.

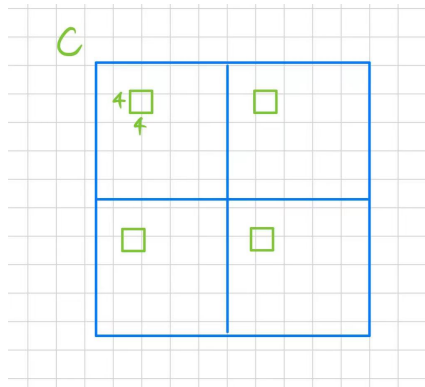


图 12

具体来说就是在 GMEM2SMEM 时:

```

182 // 使用register中转掩盖GMEM latency
183 #pragma unroll
184 for (int index = 0; index < read_per_thread; index += 4)
185 {
186     FLOAT4(A_reg_load[index]) = FLOAT4(a[a_gmem_addr + index]);
187     FLOAT4(B_reg_load[index]) = FLOAT4(b[b_gmem_addr + index]);
188 }
189 // 从register中转到SMEM
190 #pragma unroll
191 for (int index = 0; index < read_per_thread; index += 4)
192 {
193     A_smem[a_smem_x + index][a_smem_y] = A_reg_load[index];
194     A_smem[a_smem_x + index + 1][a_smem_y] = A_reg_load[index + 1];
195     A_smem[a_smem_x + index + 2][a_smem_y] = A_reg_load[index + 2];
196     A_smem[a_smem_x + index + 3][a_smem_y] = A_reg_load[index + 3];
197     FLOAT4(B_smem[b_smem_y][b_smem_x + index]) = FLOAT4(B_reg_load[index]);
198 }
199 __syncthreads();
200
201

```

图 13: GMEM2SMEM

在计算和写入时:

```

126 __global__ void conflictFreeGEMM_sn16(
204     for (int tn = 0; tn < sn; tn++)
205     {
206         // 从SMEM中加载到register
207         // 取出A的对应列和B的对应行
208         // 由于A在存入时转置了,所以在SMEM中都是按行取出
209         FLOAT4(A_reg_compute[0]) = FLOAT4(A_smem[tn][T_y * ty / 2]);
210         FLOAT4(A_reg_compute[4]) = FLOAT4(A_smem[tn][T_y * ty / 2 + sm / 2]);
211         FLOAT4(B_reg_compute[0]) = FLOAT4(B_smem[tn][T_x * tx / 2]);
212         FLOAT4(B_reg_compute[4]) = FLOAT4(B_smem[tn][T_x * tx / 2 + sm / 2]);
213
214 #pragma unroll
215 for (int tm = 0; tm < ty; tm++)
216 {
217     #pragma unroll
218     for (int tk = 0; tk < tx; tk++)
219     {
220         C_reg[tm][tk] += A_reg_compute[tm] * B_reg_compute[tk];
221     }
222 }
223
224 __syncthreads();
225
226 }
227
228 // 写回上面两块tiles的计算结果
229 #pragma unroll
230 for (int ri = 0; ri < ty / 2; ri++)
231 {
232     int store_c_gmem_m = B_y * sm + T_y * ty / 2 + ri;
233     int store_c_gmem_n = B_x * sk + T_x * tx / 2;
234     int store_c_gmem_addr = OFFSET(store_c_gmem_m, store_c_gmem_n, N);
235     FLOAT4(c[store_c_gmem_addr]) = FLOAT4(C_reg[ri][0]);
236     FLOAT4(c[store_c_gmem_addr + sk / 2]) = FLOAT4(C_reg[ri][4]);
237 }
238 // 写回下面两块tiles的计算结果
239 #pragma unroll
240 for (int ri = 0; ri < ty / 2; ri++)
241 {
242     int store_c_gmem_m = B_y * sm + T_y * ty / 2 + ri + sm / 2;
243     int store_c_gmem_n = B_x * sk + T_x * tx / 2;
244     int store_c_gmem_addr = OFFSET(store_c_gmem_m, store_c_gmem_n, N);
245     FLOAT4(c[store_c_gmem_addr]) = FLOAT4(C_reg[ri + ty / 2][0]);
246     FLOAT4(c[store_c_gmem_addr + sk / 2]) = FLOAT4(C_reg[ri + ty / 2][4]);
247 }

```

图 14: Compute

2.1.5 Double Buffer

独立出第一次 GMEM2SMEM 和最后一次计算, 中间的计算和访存两两一组, 用一个 pointer 判断计算和写入的位置, 从而异步地并行起来, 相当于用 2 倍的 SMEM 换访问 GMEM 的延迟时间. 注意 GPU 不能乱序执行即可.

SMEM:

```
29 // SMEM
30 // 注意为了减少bank conflict, 此处A按列存储, B按行存储,
31 __shared__ float A_smem[2][sn][sm];
32 __shared__ float B_smem[2][sn][sk];
33
```

图 15: SMEM

```
52 {
53     int a_gmem_x = a_smem_x;
54     int b_gmem_y = b_smem_y;
55     int a_gmem_addr = OFFSET(a_gmem_y, a_gmem_x, N);
56     int b_gmem_addr = OFFSET(b_gmem_y, b_gmem_x, K);
57 #pragma unroll
58 for (int index = 0; index < read_per_thread; index += 4)
59 {
60     FLOAT4(A_reg_load[index]) = FLOAT4(a[a_gmem_addr + index]);
61     FLOAT4(B_reg_load[index]) = FLOAT4(b[b_gmem_addr + index]);
62 }
63 #pragma unroll
64 for (int index = 0; index < read_per_thread; index += 4)
65 {
66     A_smem[0][a_smem_x + index][a_smem_y] = A_reg_load[index];
67     A_smem[0][a_smem_x + index + 1][a_smem_y] = A_reg_load[index + 1];
68     A_smem[0][a_smem_x + index + 2][a_smem_y] = A_reg_load[index + 2];
69     A_smem[0][a_smem_x + index + 3][a_smem_y] = A_reg_load[index + 3];
70     FLOAT4(B_smem[0][b_smem_y][b_smem_x + index]) = FLOAT4(B_reg_load[index]);
71 }
72 __syncthreads();
73 }
74
```

图 16: 第一次访存

```
126 #pragma unroll
127 for (int tn = 0; tn < sn; tn++)
128 {
129     // 从SMEM中加载到register
130     // 取出A的对应列和B的对应行
131     // 由于A在存入时转置了,所以在SMEM中都是按行取出
132     FLOAT4(A_reg_compute[0]) = FLOAT4(A_smem[1][tn][T_y * ty / 2]);
133     FLOAT4(A_reg_compute[4]) = FLOAT4(A_smem[1][tn][T_y * ty / 2 + sm / 2]);
134     FLOAT4(B_reg_compute[0]) = FLOAT4(B_smem[1][tn][T_x * tx / 2]);
135     FLOAT4(B_reg_compute[4]) = FLOAT4(B_smem[1][tn][T_x * tx / 2 + sm / 2]);
136 }
137 #pragma unroll
138 for (int tm = 0; tm < ty; tm++)
139 {
140     #pragma unroll
141     for (int tk = 0; tk < tx; tk++)
142     {
143         C_reg[tm][tk] += A_reg_compute[tm] * B_reg_compute[tk];
144     }
145 }
146 // 写回上面两块ciles的计算结果
147
```

图 17: 最后一次计算

3 实验结果

Algorithm	BLOCKSIZE=8	Time (s)	TFLOPS	P RATIO (%)
Matrix dims MxNxK: 128x128x128				
CUBLAS		0.000125	0.033501	100.00
naiveGEMM		0.000009	0.481351	1436.84
blockGEMM_sn8		0.000016	0.256991	767.12
blockGEMM_sn16		0.000016	0.269109	803.29
vec_GEMM_sn8		0.000014	0.290532	867.24
vec_GEMM_sn16		0.000013	0.332987	993.97
conflictFreeGEMM_sn8		0.000015	0.286816	856.15
conflictFreeGEMM_sn16		0.000013	0.326204	973.72
doubleBufferGEMM_sn8		0.000009	0.456681	1363.20
doubleBufferGEMM_sn16		0.000012	0.351296	1048.62
Matrix dims MxNxK: 256x256x256				
CUBLAS		0.000008	4.337260	100.00
naiveGEMM		0.000030	1.117879	25.77
blockGEMM_sn8		0.000031	1.070221	24.68
blockGEMM_sn16		0.000030	1.108097	25.55
vec_GEMM_sn8		0.000021	1.566535	36.12
vec_GEMM_sn16		0.000020	1.654114	38.14
conflictFreeGEMM_sn8		0.000020	1.679455	38.72
conflictFreeGEMM_sn16		0.000017	1.942491	44.79
doubleBufferGEMM_sn8		0.000013	2.624031	60.50
doubleBufferGEMM_sn16		0.000016	2.097068	48.35
Matrix dims MxNxK: 512x512x512				
CUBLAS		0.000028	9.611531	100.00
naiveGEMM		0.000187	1.436403	14.94
blockGEMM_sn8		0.000071	3.770602	39.23
blockGEMM_sn16		0.000071	3.768882	39.21
vec_GEMM_sn8		0.000044	6.057216	63.02
vec_GEMM_sn16		0.000045	6.003441	62.46
conflictFreeGEMM_sn8		0.000038	7.005276	72.88
conflictFreeGEMM_sn16		0.000037	7.253005	75.46
doubleBufferGEMM_sn8		0.000029	9.237029	96.10
doubleBufferGEMM_sn16		0.000037	7.229845	75.22

表 1: Performance for matrix dimensions and block dims 8x8

Algorithm	BLOCKSIZE=8	Time (s)	TFLOPS	P RATIO (%)
Matrix dims MxNxK: 1024x1024x1024				
CUBLAS		0.000143	14.970034	100.00
naiveGEMM		0.001411	1.522147	10.17
blockGEMM_sn8		0.000401	5.360565	35.81
blockGEMM_sn16		0.000400	5.368544	35.86
vec_GEMM_sn8		0.000202	10.647249	71.12
vec_GEMM_sn16		0.000226	9.502503	63.48
conflictFreeGEMM_sn8		0.000157	13.674686	91.35
conflictFreeGEMM_sn16		0.000171	12.592068	84.12
doubleBufferGEMM_sn8		0.000145	14.773098	98.68
doubleBufferGEMM_sn16		0.000205	10.485686	70.04
Matrix dims MxNxK: 2048x2048x2048				
CUBLAS		0.001060	16.214812	100.00
naiveGEMM		0.012010	1.430447	8.82
blockGEMM_sn8		0.003149	5.455887	33.65
blockGEMM_sn16		0.003113	5.518230	34.03
vec_GEMM_sn8		0.001538	11.170883	68.89
vec_GEMM_sn16		0.001702	10.094356	62.25
conflictFreeGEMM_sn8		0.001173	14.648097	90.34
conflictFreeGEMM_sn16		0.001302	13.194517	81.37
doubleBufferGEMM_sn8		0.001130	15.203838	93.77
doubleBufferGEMM_sn16		0.001314	13.073310	80.63

表 2: Performance for matrix dimensions and block dims 8x8

Algorithm	BLOCKSIZE=16	Time (s)	TFLOPS	P RATIO (%)
Matrix dims MxNxK: 128x128x128				
CUBLAS		0.000133	0.031637	100.00
naiveGEMM		0.000011	0.390945	1235.73
blockGEMM_sn8		0.000028	0.148554	469.56
blockGEMM_sn16		0.000028	0.148030	467.90
vec_GEMM_sn8		0.000020	0.207654	656.37
vec_GEMM_sn16		0.000019	0.225163	711.72
conflictFreeGEMM_sn8		0.000015	0.270746	855.80
conflictFreeGEMM_sn16		0.000014	0.290416	917.97
doubleBufferGEMM_sn8		0.000014	0.306755	969.62
doubleBufferGEMM_sn16		0.000013	0.327639	1035.63
Matrix dims MxNxK: 256x256x256				
CUBLAS		0.000008	4.380657	100.00
naiveGEMM		0.000029	1.173691	26.79
blockGEMM_sn8		0.000046	0.725588	16.56
blockGEMM_sn16		0.000046	0.725262	16.56
vec_GEMM_sn8		0.000035	0.961917	21.96
vec_GEMM_sn16		0.000033	1.022348	23.34
conflictFreeGEMM_sn8		0.000029	1.141916	26.07
conflictFreeGEMM_sn16		0.000028	1.185897	27.07
doubleBufferGEMM_sn8		0.000024	1.404280	32.06
doubleBufferGEMM_sn16		0.000022	1.497998	34.20
Matrix dims MxNxK: 512x512x512				
CUBLAS		0.000028	9.487658	100.00
naiveGEMM		0.000165	1.628943	17.17
blockGEMM_sn8		0.000079	3.393333	35.77
blockGEMM_sn16		0.000081	3.323333	35.03
vec_GEMM_sn8		0.000059	4.541440	47.87
vec_GEMM_sn16		0.000057	4.745721	50.02
conflictFreeGEMM_sn8		0.000052	5.175087	54.55
conflictFreeGEMM_sn16		0.000049	5.507335	58.05
doubleBufferGEMM_sn8		0.000044	6.169342	65.02
doubleBufferGEMM_sn16		0.000043	6.300402	66.41

表 3: Performance for matrix dimensions and block dims 8x8

Algorithm	BLOCKSIZE=16	Time (s)	TFLOPS	P RATIO (%)
Matrix dims MxNxK: 1024x1024x1024				
CUBLAS		0.000136	15.787256	100.00
naiveGEMM		0.001130	1.899829	12.03
blockGEMM_sn8		0.000268	8.009450	50.73
blockGEMM_sn16		0.000269	7.972631	50.50
vec_GEMM_sn8		0.000210	10.223862	64.76
vec_GEMM_sn16		0.000201	10.696234	67.75
conflictFreeGEMM_sn8		0.000171	12.543760	79.45
conflictFreeGEMM_sn16		0.000161	13.323657	84.40
doubleBufferGEMM_sn8		0.000151	14.199013	89.94
doubleBufferGEMM_sn16		0.000145	14.813126	93.83
Matrix dims MxNxK: 2048x2048x2048				
CUBLAS		0.001023	16.792684	100.00
naiveGEMM		0.009086	1.890818	11.26
blockGEMM_sn8		0.001568	10.956499	65.25
blockGEMM_sn16		0.001625	10.573978	62.97
vec_GEMM_sn8		0.001234	13.921037	82.90
vec_GEMM_sn16		0.001249	13.750360	81.88
conflictFreeGEMM_sn8		0.001097	15.658899	93.25
conflictFreeGEMM_sn16		0.001043	16.466607	98.06
doubleBufferGEMM_sn8		0.000980	17.538562	104.44
doubleBufferGEMM_sn16		0.000943	18.214506	108.47

表 4: Performance for matrix dimensions and block dims 16x16

GPU 信息:

```

1  Device 0: NVIDIA GeForce RTX 4070
2  Total number of SMs: 46
3  Maximum number of threads per SM: 1536
4  Maximum number of threads per block: 1024
5  Maximum size of each dimension of a block: 1024 x 1024 x 64
6  Maximum size of each dimension of a grid: 2147483647 x 65535 x 65535
7  Shared memory per block: 49152 bytes
8  Total global memory: 11.9937 GB
9  Number of registers per SM: 65536
10 Number of registers per block: 65536
11 Maximum registers per thread: 64

```

图 18: RTX4070

BLOCK SIZE = 8:

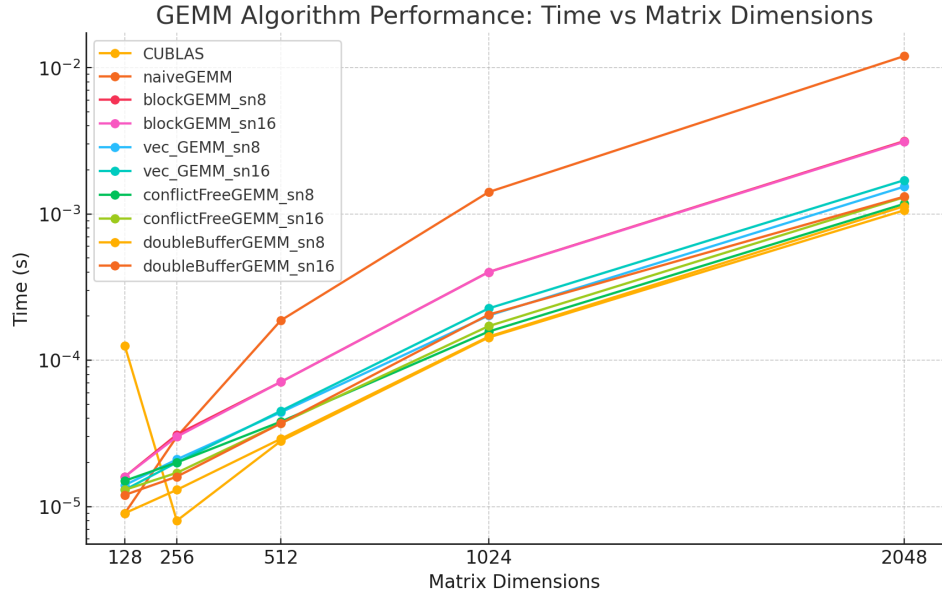


图 19: BLOCK SIZE = 8, 时间, log scale

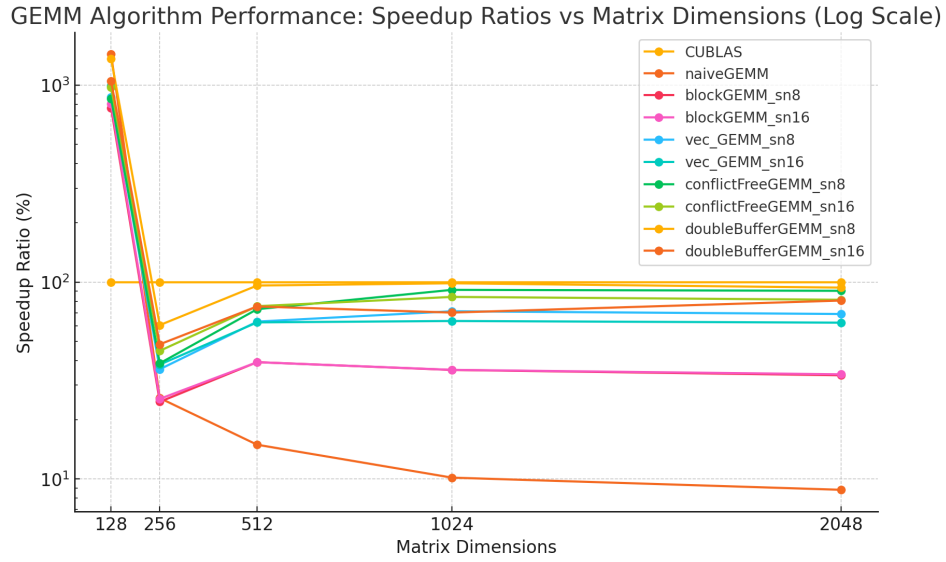


图 20: BLOCK SIZE = 8, 加速比, log scale

BLOCK SIZE = 16:

GEMM Algorithm Performance (16x16 Blocks): Time vs Matrix Dimensions (Log Scale)

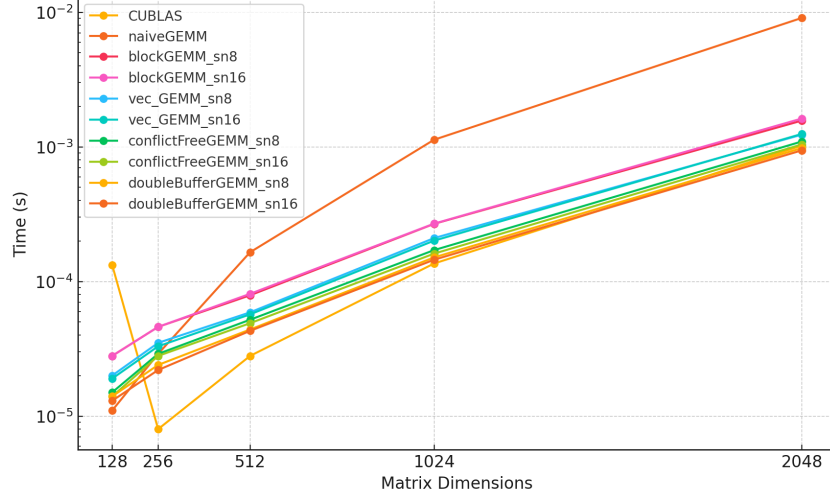


图 21

GEMM Algorithm Performance (16x16 Blocks): Speedup Ratios vs Matrix Dimensions (Log Scale)

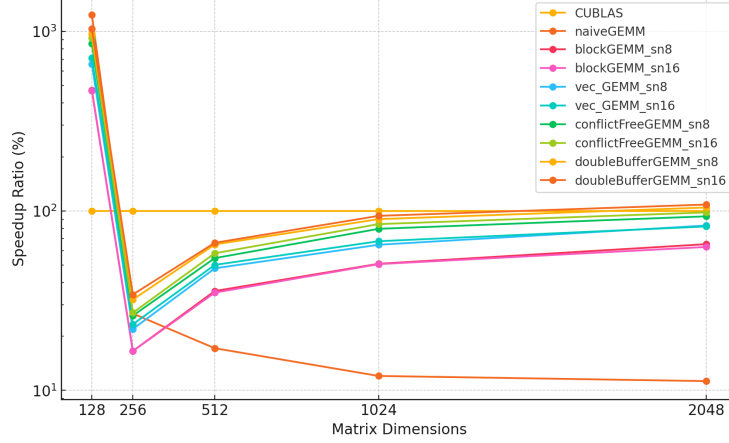


图 22

实验结果分析

1. 使用-maxrregcount=128 命令编译, 确保编译出的版本最多使用 128 个寄存器, 否则会编译出 129 或 133 寄存器的版本, 降低 occupancy.
2. 可以看到和2.1.2处的理论分析一致,BLOCKSIZE=16 确实是性能最优的 (基本全面优于 BLOCKSIZE=8). 每种优化基本都提高了算法的性能, 其中 BLOCK SIZE=16 时的 doubleBufferGEMM_sn=16 性能最好, 在大型矩阵 (≥ 2048) 上

的性能基本超过了 cublas(在 4070 上测试均超过 cublas,4090 上约有 cublas90-95 的性能).

3. 分块算法显著地提高了性能 ($\times 2\text{-}\times 10$). 在大型稠密矩阵上性能大致上是 `doubleBufferGEMM_sn=16` > `doubleBufferGEMM_sn=8` > `conflictFreeGEMM_sn=16` > `conflictFreeGEMM_sn=8` > `vec_GEMM_sn=8` > `vec_GEMM_sn=16` > `blockGEMM_sn=8` > `blockGEMM_sn=16` > `naiveGEMM`. 小型矩阵 (128) 上, 大致是: `naiveGEMM` > `doubleBufferGEMM_sn=16` > `doubleBufferGEMM_sn=8` > `conflictFreeGEMM_sn=16` > `conflictFreeGEMM_sn=8` > `vec_GEMM_sn=16` > `vec_GEMM_sn=8` > `blockGEMM_sn=8` > `blockGEMM_sn=16`
4. 同时可以发现在中型矩阵 (256-1024) 下, cublas 的性能仍然是最好的, 这可能是由于我们没有对 register 的 bank conflict 做处理.
5. 在小型矩阵 (128) 下 naive 算法反而是性能最好的, 这可能使由于其实现简单, 访存和 register 使用较少, 提高了 occupancy.
6. sn=16 的算法在处理了 bank conflict 后性能优于 sn=8, 这是因为它减少了迭代次数, 但增加了 bank conflict 的发生次数 (8 次->16 次).
7. 在所有算法中 (除了 CUBLAs), 时间开销基本和矩阵大小成指数关系 (对数图上呈直线).
8. 最终结果会有 $1e-3$ 数量级的 rounding error.

4 实验感想

这次实验让我深入理解了并行计算中访存优化和块大小选择的重要性。通过调整块大小、优化访存方式, 以及避免寄存器和共享内存的冲突, 显著提升了矩阵乘法的性能。具体来说, 在选择适当的块大小时, 我们考虑了寄存器和共享内存的使用情况, 确保每个线程块内的线程数不会超过硬件限制, 同时最大化寄存器的利用率。通过将数据从全局内存移动到共享内存, 然后再从共享内存移动到寄存器, 我们有效地减少了访存延迟, 提高了计算效率。